

1. Framework Architecture Overview



User Interface Layer:

- Uses OMEdit for equation-based model definition (Modelica)
- Uses OpenSim GUI for visualization of OpenSim Model
- Configuration files:
 - File paths to FMUs, OpenSim models, FEM models
 - Defining values for parameters and initial values
 - Simulation settings: start/stop time, step size, solver settings
 - Logging and output options
- Jupyter Notebook: used as interactive orchestration environment

Core Layer:

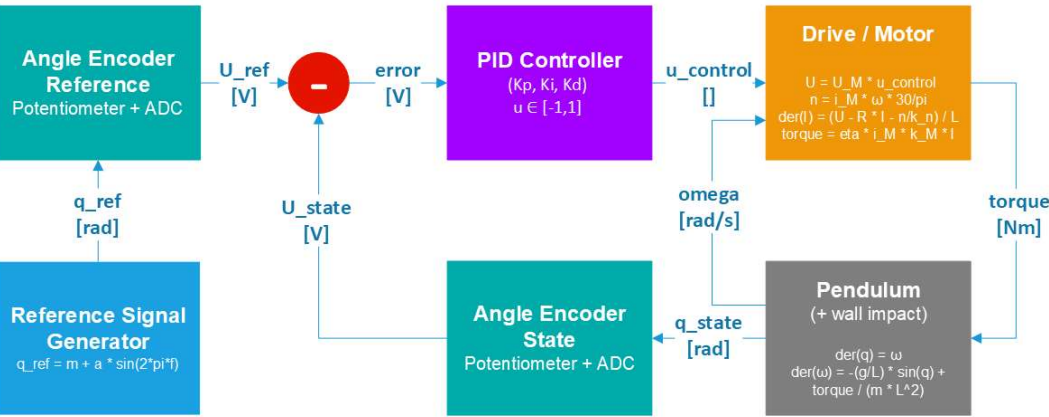
- Core layer between the user definitions and the solver / model backends
- Provides a unified interface protocol for simulation components
- Simulation master orchestrates the execution of the subsystem models
- Configuration file is parsed, and corresponding system model is set up, connections are validated

Backend Layer:

- Layer represents the heterogeneous simulation engines
- Each backend is accessed exclusively through a wrapper, ensuring unified protocol compliance
- Equation-based models are built in OpenModelica and exports as Co-Simulation FMUs via OMPython
- FMUs are imported and simulated via FMPy (solver is inside the FMU)
- OpenSim models are simulated using opensim Python API
- FEM models could be solved via openCFS / CalculiX, interface via simulation file manipulation and subprocess execution (file-based adapters)
- **FEM: Proof of concept planned**

2. Demo Model: Controlled Pendulum

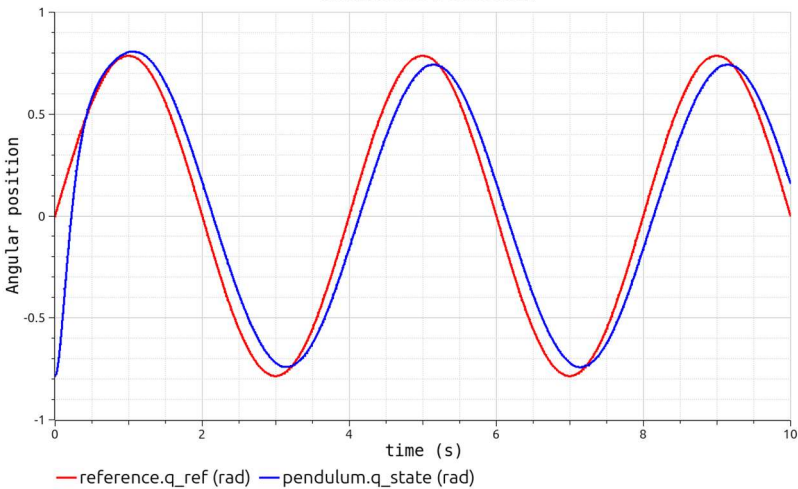
Simplified Block Diagram:



Suitability as Framework Demonstrator:

- **Simple benchmark** system with mechanical, electrical, and control subsystems
- **Captures core challenges of system simulation:**
 - Continuous + discrete dynamics
 - Sensors + quantization
 - Feedback control
 - Heterogeneous model integration
- **Provides a progression of complexity:**
 1. Equation-based pendulum + controller (FMUs)
 2. Hybrid extension (Impact Wall = discrete event)
 3. Replacement Pendulum with OpenSim model
 4. Integration with FEM (elastic rod/contact with wall)
- Serves as **proof-of-concept for framework capabilities**

Controlled Pendulum



Subsystem	Description
Reference Signal Generator	- Creates a sine signal oscillating around a mean with defined amplitude and frequency - Provides the reference input for the control loop (ideal trajectory)
Angle Encoder	- Maps the pendulum's angular position into an analog voltage using a potentiometer model - Adds a quantization via ADC with $nBits$ resolution and clips to ADC voltage range
PID Controller	- Compares the reference signal (U_{ref}) and state signal (U_{state}) to compute an error - Implements the PID law (continuous and sample form implemented) - Outputs a control signal in the range -1 to 1 which drives the motor voltage
Drive / Motor	- Computes the current motor voltage based on the controller output - Computes the resulting current dynamics using inductance and resistance - Provides the actuation torque acting to the pendulum hinge joint
Pendulum	- Models a 1-DOF mechanical pendulum: rigid massless rod of length L with a point mass m at its end - Takes torque input from the drive, outputs angular position and velocity

(Sima, 2024)

6. Demo Model Extension: Pendulum with Wall Impact

OpenSim: Hunt Crossley Contact

- Contact modeled as compliant sphere–plane contact
- Directly supported in OpenSim via **HuntCrossleyForce**
- Natural biomechanics connection
- Handles **continuous impact with damping**
- Demonstrates ability to replace eq.-based pendulum with biomechanical one
- Shows hybrid integration: OpenSim + FMU in one loop
- **Limitation:** more complex parameterization (contact geometry, stiffness, damping)
- **Advantage:** can export motion files for visualization in OpenSim GUI.

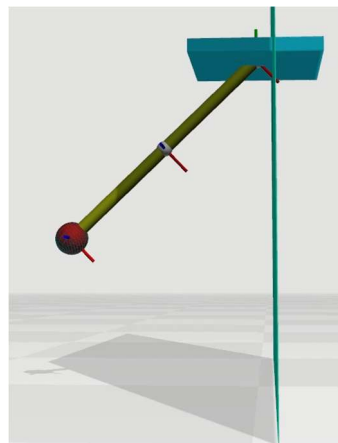
PrePoMax / CalculiX (FEM impact)

- **Static Analysis:**
 - Hertz contact (sphere–plane, hard formulation).
 - Extracts stiffness curve (force vs. penetration).
 - Allows stress/strain fields in contact region
- **Dynamic Analysis:**
 - Implicit Dynamic Simulation
 - Surface interaction: linear-pressure overclosure
- Shows true advantage of FEM integration: **localized stresses and deformations**
- But: limited Python scripting in PrePoMax, integration not trivial
- **Candidates for future:**
 - CalculiX standalone with Python editing of input decks
 - openCFS (no implicit contact modeling, available via Nitsche Interface Non-Conforming Interface)

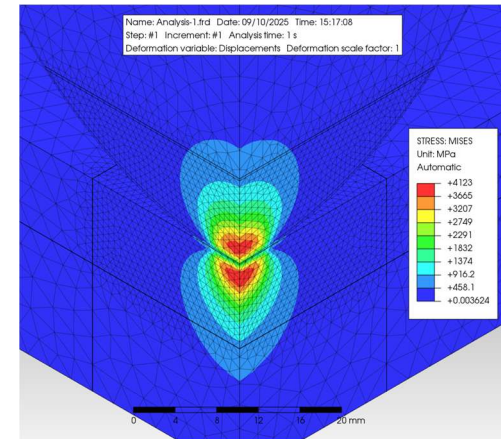
HuntCrossleyForce Definition

Properties	
actions (?)	
name (?)	HuntCrossleyForce
components (?)	(No Objects)
appliesForce (?)	☒
geometry (?)	wall_contact  head_contact  + Add Entry
stiffness	100000.000000 - +
dissipation	0.500000 - +
static_friction	0.900000 - +
dynamic_friction	0.900000 - +
viscous_friction	0.600000 - +
transition_velocity (?)	0.010000 - +

OpenSim Visualization



Static: Hertzian Contact



Dynamic Impact

